Can a Model Tell Cause from Correlation? A Simple Pipeline on the Corr2Cause Dataset

Chaitanya Sai Meka, Rakshita, Dhanvin Vadlamudi, Kasula Lalithendra, Jagruthi Pulumati
CSAI Project, Phase 1 (Machine Learning)
Code: https://github.com/Jag2007/AI_ML_Project

***Abstract*—"Two things move together" does not mean "one causes the other". The Corr2Cause dataset tests exactly this. Each example gives a short text (the premise) that says which variables are related, and a claim (the hypothesis) such as "A directly causes B". The task is to say if the claim is surely true (1) or not (0). We built a five-step pipeline, one notebook per team member: clean the data, turn the text into a causal graph with the PC algorithm, build features, train classifiers, and finally build a DAG and ask it "what if" (intervention) questions with a Bayesian network. With text features (TF-IDF) alone, logistic regression gets an F1 of only 0.47. Adding features from the causal graph raises it to 0.88. The graph step alone, with no learning, reproduces 98.5% of the test labels. Our final model, a Random Forest, gets a test F1 of 0.957. 89% of the cause arrows it draws are correct, and its graphs answer 92 of 96 intervention questions correctly.**

I. Introduction

The problem. Ice cream sales and drowning cases both go up in summer. They are *correlated*, but ice cream does not cause drowning; hot weather causes both. Large language models often make this mistake and treat correlation as causation.

The dataset. Corr2Cause [1] turns this into a yes/no task. A real example (shortened):

> *Premise:* There is a closed system of 3 variables, A, B and C. A correlates with B. A is independent of C. B is independent of C.
> *Hypothesis:* There is a common effect (collider) of A and C.
> *Label:* 0 (not valid)

The premise says which pairs are related ("correlates") and which are not ("independent"). The hypothesis is a cause-and-effect claim. The label is 1 only if the claim is true in *every* causal graph that fits the premise. If even one fitting graph breaks the claim, the label is 0.

Our goal. We did not want a black box. We wanted every prediction to come from a graph we can draw and explain. So we combined a classic causal discovery method (the PC algorithm) with simple machine learning models. This is a *neuro-symbolic* design: the learned part reads the text, and the symbolic part (the graph) does the exact reasoning.

II. Related Work

Causal graphs. Pearl [2] showed how to reason about cause and effect with directed acyclic graphs (DAGs) and Bayesian networks. Verma and Pearl [3] showed that different DAGs can give the same independences when they have the same skeleton (links without direction) and the same colliders. So from independences alone we can only find a group of DAGs, called the *Markov equivalence class* (MEC). The PC algorithm [4] finds this class from independence facts, and Meek's rules [5] set every arrow direction that is forced.

TABLE I
Earlier work and the gap we address

Work	What it does	What is missing
PC algorithm [4]	Finds a causal graph from data	Cannot read text
Text extraction [6], [7], [8]	Finds causal phrases	No check that the claim follows
LLMs on Corr2Cause [1]	Answer yes/no end to end	Near chance, or breaks on rewording
Ours	Parser → PC → features → classifier → DAG	Every answer comes from a graph

Finding causes in text. Early systems found cause–effect pairs with hand-written patterns [6]. SemEval-2010 Task 8 [7] and BECauSE 2.0 [8] label causal phrases in sentences. These works find *what a text claims*. They do not check if the claim *follows* from the evidence, and they do not build a graph.

Causal reasoning tests for language models. The Corr2Cause authors [1] found that 17 large language models score close to chance. Fine-tuned models score well, but fail when the sentences are reworded or the variable names are changed, so they learn surface patterns, not reasoning. CLadder [9] asks similar causal questions, but only tests models and does not give a pipeline.

Neuro-symbolic AI. These systems let a learned part read the input and a rule-based part do exact reasoning [10]. Our pipeline follows this idea. Table I sums up the gap we fill.

III. Key Ideas

DAG. A directed acyclic graph: arrows mean "directly causes", and you can never follow the arrows back to where you started (no cycles, no self-loops).

Collider and confounder. In X→Z←Y, Z is a *collider* (common effect). In X←Z→Y, Z is a *confounder* (common cause).

Conditional independence. "A and C are independent given B" means that once we know B, knowing A tells us nothing more about C. The premises list these facts, and the PC algorithm uses them.

TABLE II
NUMBER OF ROWS BEFORE AND AFTER CLEANING

Split	Before	After	Premises
Train (sampled)	205,734	35,340	400
Validation	1,076	1,026	19
Test	1,162	1,162	21

Markov equivalence class and mec_support. Many DAGs can fit one premise. For each claim we count in how many of them it is true:

$$\text{mec_support} = \frac{\text{fitting DAGs where the claim is true}}{\text{all fitting DAGs}}. \quad (1)$$

If mec_support $= 1$, the claim holds in every fitting DAG, so it should be valid. We call "predict valid when mec_support $= 1$" the *symbolic rule*. It uses no learning.

Intervention. Forcing a variable to a value is written $do(X)$. This is different from just observing X. We measure the effect of X on Y as

$$\text{effect} = P(Y{=}1 \mid do(X{=}1)) - P(Y{=}1 \mid do(X{=}0)). \quad (2)$$

If Y is not downstream of X in the graph, the effect is 0.

Evaluation. From the confusion matrix (TP, FP, FN, TN):

$$P = \frac{TP}{TP+FP}, \qquad R = \frac{TP}{TP+FN}, \qquad F_1 = \frac{2PR}{P+R}. \quad (3)$$

Precision (P): of the claims we called valid, how many really were. Recall (R): of the truly valid claims, how many we found. F1 balances the two.

IV. STEP 1: CLEANING THE DATA (NOTEBOOK 01)

We used the official train, validation and test splits from Hugging Face (Table II). Each row has the text (`input`), the `label`, the number of variables, and a `template` that says which type of claim it is.

Checking. We looked at `head()`, `tail()`, the shape and the column types, and counted missing values and duplicates. There were none, so nothing had to be filled in or dropped.

Cleaning. Every text has exactly one "Hypothesis:" marker, so we split it into a `premise` and a `hypothesis` column and removed extra spaces. We checked that the number of variables written in the premise matches the `num_variables` column. The same premise is reused with up to 90 claims, so we gave each premise an ID.

Removing data leakage. A few premises were in two splits at once (2 in train and validation, 3 in validation and test). If the model sees a premise in training and again in testing, its score looks better than it really is. We kept the test set as it was and removed the shared premises from train and validation.

Subsampling train. Train has about 206,000 rows, which is slow. We randomly picked 400 whole premises (seed 42) with all their claims, giving 35,340 rows. The share of valid claims stayed the same (18.5%).

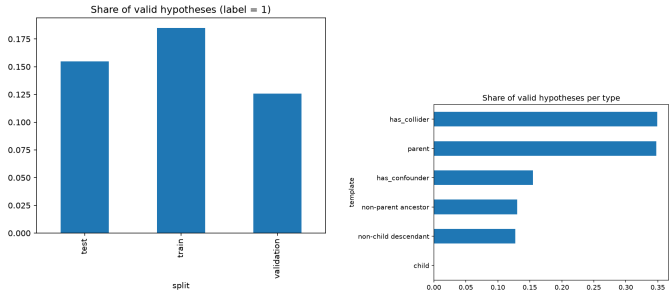


Fig. 1. Left: only 13–19% of claims are valid in each split. Right: share of valid claims for each claim type in train. "Child" claims are never valid.

V. EXPLORATORY DATA ANALYSIS

Class imbalance. Only 13–19% of claims are valid (Fig. 1, left). A model that always says "not valid" would already be over 80% accurate, so accuracy is misleading. We use precision, recall and F1 (Eq. 3), and we set `class_weight="balanced"` so the rare valid class counts as much as the common one.

A hidden shortcut. "Child" claims are never valid (Fig. 1, right), yet their text ("X directly causes Y") is the same as "parent" claims. So the `template` column gives away the label. We never use it as a feature, only to study mistakes.

Harder test set. Train only has graphs with 5–6 variables, but validation and test also have 2–4 variables. So the test set has cases the model never saw.

VI. STEP 2: TEXT TO GRAPH (NOTEBOOK 02)

Parsing. The sentences follow fixed templates, so regular expressions (regex) read them exactly. From each claim we take the two variables (the single capital letters) and the claim type (direct cause, indirect cause, collider, confounder) from keywords. We added schema checks: variables must be A–F and appear in the premise, no self-loops (A→A), and no unknown wording. All real rows passed, and broken examples we wrote by hand were caught.

PC algorithm. PC normally tests independence on numeric data. Here the premise already states the independences, so pgmpy's PC reads them directly (`ci_test="independence_match"`). PC (1) starts with every pair linked and removes the link between independent pairs, (2) finds colliders X→Z←Y, and (3) uses Meek's rules to set more directions. Links it cannot decide stay undirected (Fig. 2).

All fitting DAGs. For each undirected link we try both directions. We keep every version that has no cycle and is I-equivalent to the PC graph (same skeleton and colliders). This gives every DAG in the Markov equivalence class, between 1 and 144 per premise. For each claim we then compute mec_support (Eq. 1).

What does each claim mean? Our first definitions were the textbook ones: a collider is a shared direct child, a confounder is a shared direct parent, and "X causes something else which causes Y" means Y is downstream of X but not a direct child.

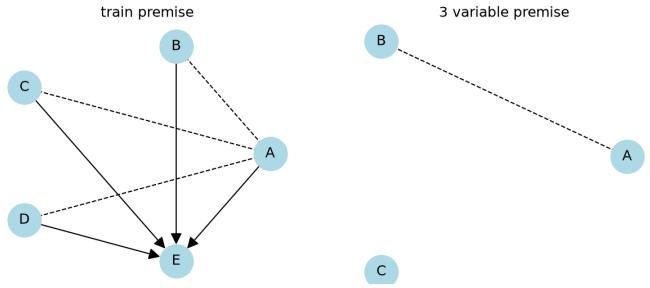


Fig. 2. PC output for two premises. Solid arrows have a known direction; dashed lines could point either way. Left: E is a common effect of A–D. Right: only A and B are linked, and C stands alone.

TABLE III
CLAIM DEFINITIONS USED BY THE CORR2CAUSE GENERATOR

Claim	Path between X and Y
X directly causes Y	X→Y
X causes sth. else which causes Y	X→ · · · →Y (2+ arrows)
Collider (common effect)	X→ · · · →Z← · · · ←Y
Confounder (common cause)	X← · · · ←Z→ · · · →Y

To match the labels better, we read the code that made the dataset [11]. It looks at every path between X and Y and names it by its arrows (Table III). The main difference: "X causes something else which causes Y" only needs a path of two or more arrows, even if X→Y also exists.

How well does the graph explain the labels? Table IV shows how often the symbolic rule (mec_support = 1) matches the label, with no learning at all.

The largest group of remaining mismatches (511 of 1,235, or 41%) are "parent" claims where every fitting DAG has X→Y but the label is 0. A public audit of the test split also reports wrong labels, so we treat these as label noise.

VII. STEP 3: FEATURE ENGINEERING (NOTEBOOK 03)

A model needs numbers, so we turn each claim into a row of features.

Structured features (15). The claim type, one-hot encoded (the types have no order); the number of correlations and independences in the premise; whether X and Y correlate, are independent, or are independent given other variables; whether PC drew X→Y, Y→X or an undirected X–Y; the number of fitting DAGs on a log scale (`log_mec_size`); and mec_support. There were no missing values. The columns have different ranges, so we standardised them with `StandardScaler`: each value becomes (value − mean) / standard deviation, with the mean and standard deviation taken from train only.

Text features. TF-IDF gives each word or word pair (unigrams and bigrams) a score: high if it is common in this text but rare in the others. We made one TF-IDF table for the premise (135 terms) and one for the hypothesis (106 terms), fitted on train only. We kept one-letter words because A–F are the variable names.

Feature selection. We ranked text terms with the chi-squared test and structured features with mutual information (how much a feature tells us about the label, Fig. 3). mec_support is far ahead, followed by the arrow-direction features. The top text terms are words like "collider" and variable letters, which say nothing about the pair being asked about. Keeping only the top 50 + 30 text terms changed F1 by about 0.02 (0.88 vs 0.90). With only 19 validation premises this is within the noise, so we kept all terms.

TABLE IV
SHARE OF LABELS MATCHED BY THE SYMBOLIC RULE

Definitions	Train	Validation	Test
Our first (textbook)	93.7%	96.5%	96.2%
Generator's	**96.6%**	**97.5%**	**98.5%**

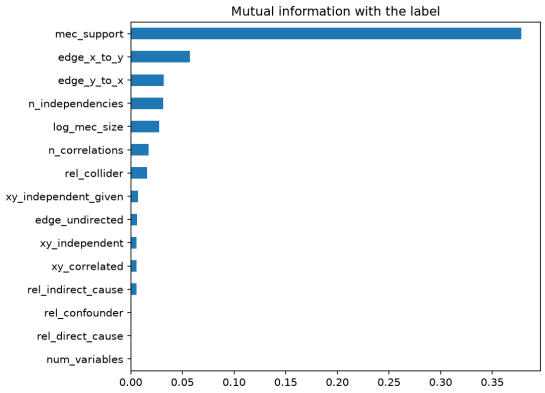


Fig. 3. Mutual information of each structured feature with the label (train). mec_support, which comes from PC, is clearly the most useful.

Text vs graph features (ablation). Table V trains the same logistic regression on three feature sets. With text only, the model calls almost everything valid: recall 0.95 but precision only 0.31. Graph features fix this, and using both is best among the learned models. The symbolic rule still scores higher (0.91): a linear model with balanced class weights cannot copy the sharp step at exactly mec_support = 1. The tree models in Step 4 can.

Both models agree (Fig. 4): the graph features, not the words, drive the predictions.

VIII. STEP 4: MODELS AND EVALUATION (NOTEBOOK 04)

Setup. Each claim becomes 256 features (135 + 106 TF-IDF + 15 structured). We tried logistic regression, Random Forest (300 decision trees that vote) and gradient boosting (small trees added one by one, each fixing the last one's mistakes). We tuned the hyperparameters with `GridSearchCV` and 5-fold cross-validation, scored by F1. We used `GroupKFold`, which keeps all claims of one premise in the same fold, so the model is never checked on a premise it trained on. Table VI shows the settings tried and the best ones.

We picked the model with the best validation F1 and used the test set only once, at the end.

Results. Random Forest had the best validation F1, so we chose it (Table VII). It is a bit better than the symbolic rule on validation (0.929 vs 0.906) and test (0.957 vs 0.952), but

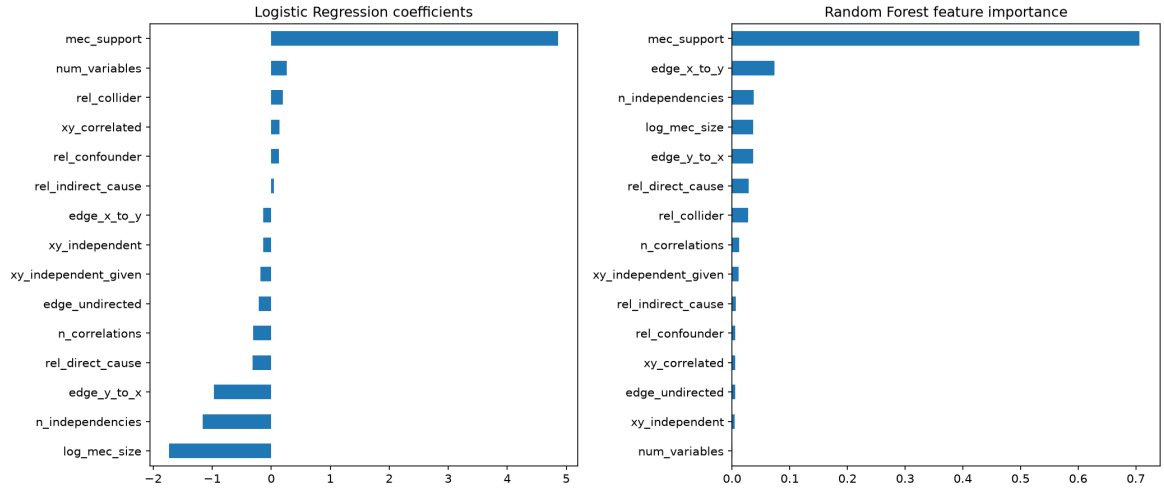


Fig. 4. Left: logistic regression coefficients. mec_support pushes strongly towards "valid" (+4.9); a large equivalence class, many independences and a Y→X arrow push towards "not valid". Right: Random Forest feature importance, where mec_support alone takes 0.71 of the total.

TABLE V
WHICH FEATURES HELP? (LOGISTIC REGRESSION, VALIDATION SET)

Features	Precision	Recall	F1
Text (TF-IDF) only	0.311	0.953	0.469
Structured only	0.840	0.853	0.846
TF-IDF + structured	0.884	0.884	**0.884**
Symbolic rule (no learning)	0.846	0.977	0.906

TABLE VI
HYPERPARAMETER TUNING (5-FOLD GROUPKFOLD ON TRAIN)

Model	Settings tried (best in bold)	CV F1
Logistic Regression	C: 0.1, 1, **10**	0.926
Random Forest	max_depth: None, **20**; min_samples_leaf: **1**, 5	0.935
Gradient Boosting	learning_rate: 0.05, **0.1**; max_leaf_nodes: 15, **31**	0.949

TABLE VII
RESULTS (VALIDATION F1; TEST PRECISION, RECALL AND F1)

Model	Val F1	Test P	Test R	Test F1
Logistic Regression	0.878	0.922	0.983	0.952
Random Forest (chosen)	**0.929**	0.927	0.989	**0.957**
Gradient Boosting	0.879	0.927	0.989	0.957
Symbolic rule (no learning)	0.906	0.922	0.983	0.952

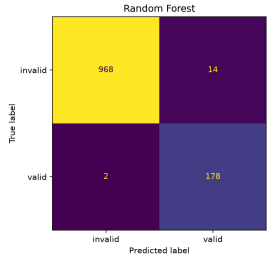


Fig. 5. Random Forest on test: 968 true negatives, 178 true positives, 14 false positives and 2 false negatives.

the test gap is inside the confidence interval below. So the graph step does almost all of the work, and the learned model mainly tidies up the edges of the rule.

Confidence interval. The test set has only 21 premises. Rows of the same premise are not independent, so we used a bootstrap over premises: we resampled whole premises 1,000 times and recomputed F1 each time. The 95% confidence interval (2.5th to 97.5th percentile) is 0.921 to 0.982.

Error analysis. The model made 16 mistakes out of 1,162 (Fig. 5). Half are false positives on "parent" claims, the same label noise found in Step 2. It made no mistakes on "child" claims. Graphs with 2–4 variables, which never appeared in training, had 1 mistake in 126 rows, about as low as the 5–6 variable graphs. This is because the structured features come from the parser and PC, not from memorised sentences.

IX. STEP 5: DAG AND BAYESIAN NETWORK (NOTEBOOK 05)

Building the DAG. For each test premise, every "X directly causes Y" claim predicted valid becomes an arrow. We add the most confident arrows first. An arrow is skipped if it is a self-loop, uses a variable not in the premise, or would create a cycle. All 21 test graphs are proper DAGs.

Edge precision. Edge precision = correct arrows / arrows drawn, and edge recall = correct arrows / true arrows. We drew 73 arrows and 65 are correct (**edge precision 0.89**), and no true arrow was missed (edge recall 1.00). Against the PC graph the edge precision is 1.00: all 8 wrong arrows are ones PC supports but the dataset labels as 0 (Fig. 6).

Bayesian network. A Bayesian network is a DAG with a probability table (CPD) at each node. Corr2Cause has no

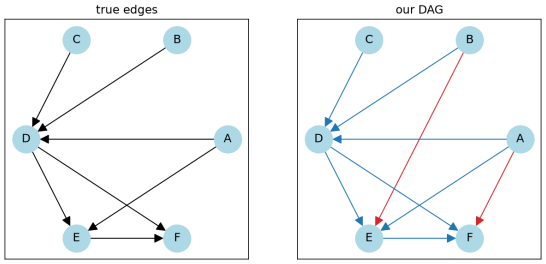


Fig. 6. One test premise. Left: the true arrows. Right: our predicted DAG. Blue arrows are correct; the red arrows are extra ones that PC supports but the dataset marks as not valid.

TABLE VIII
INTERVENTION QUERIES ON THE FIVE FULLY KNOWN TEST GRAPHS

Graph	All queries	Real effects	Mean |error|
Our predicted DAG	**92 / 96**	**28 / 32**	**0.011**
Same DAG, arrows reversed	36 / 96	0 / 32	0.126
Empty graph	64 / 96	0 / 32	0.064

numbers, so we made some: we gave the true graph random probability tables, simulated 5,000 rows from it, and fitted our DAG on those rows with pgmpy. Then we measured intervention effects with Eq. (2).

Why direction matters. For the chain A→B→C, the true effect of A on C is 0.490, and the correct graph fitted on the data gives 0.498. The reversed graph C→B→A fits the same data but gives 0.000 for A on C, and wrongly says that C affects A (0.514).

Intervention queries. We used the five test premises where PC knows the full graph (only one DAG in the class). For every ordered pair of variables we asked "what is the effect of $do(X)$ on Y?": 96 queries, 32 with a real effect. A query is correct only if the graph gets both (1) whether there is an effect at all (a directed path) and (2) its size within 0.05. We compare our DAG with two baselines fitted on the same data (Table VIII).

Our DAGs get 28 of the 32 real effects right; both baselines get none. So getting the arrows right, in the right direction, is what makes the "what if" answers right.

X. Conclusion and Next Steps

Turning the text into a graph with a regex parser and the PC algorithm explains 97–98% of the labels with no learning. Adding these graph features to TF-IDF and training a Random Forest gives a test F1 of 0.957 (95% CI 0.921–0.982), with no data leakage between splits. TF-IDF alone reaches only 0.47, which shows that word patterns cannot replace causal reasoning. Most remaining mistakes come from a small set of labels that disagree with PC.

Limitations. Corr2Cause is made from templates, so our regex parser would not work on reworded or real text. Train only has 5–6 variable premises. The Bayesian network data is simulated, not measured.

Phase 2. We plan to replace the regex parser and TF-IDF with a fine-tuned transformer that can read reworded text, keep PC for the reasoning, and test on reworded and renamed-variable versions of the data, where the Corr2Cause authors found fine-tuned models break down.

Contribution Matrix

Each person owned one notebook (✓ = owner).

Task	Chaitanya	Rakshita	Dhanvin	Lalithendra	Jagruthi
Loading and checking data	✓				
Cleaning, leakage, sampling	✓				
EDA and plots	✓				
Text parser and safety checks		✓			
PC algorithm, mec_support		✓			
Building features			✓		
Feature selection, text vs graph			✓		
Model tuning and selection				✓	
Evaluation, error bars, mistakes				✓	
Building the DAG, arrow precision					✓
Bayesian network, "what if"					✓
Report and repository					✓

References

[1] Z. Jin, J. Liu, Z. Lyu, S. Poff, M. Sachan, R. Mihalcea, M. Diab, and B. Schölkopf, "Can large language models infer causation from correlation?" in *Proc. ICLR*, 2024.

[2] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge Univ. Press, 2009.

[3] T. Verma and J. Pearl, "Equivalence and synthesis of causal models," in *Proc. UAI*, 1990, pp. 255–268.

[4] P. Spirtes, C. Glymour, and R. Scheines, *Causation, Prediction, and Search*, 2nd ed. MIT Press, 2000.

[5] C. Meek, "Causal inference and causal explanation with background knowledge," in *Proc. UAI*, 1995, pp. 403–410.

[6] C. S. G. Khoo, S. Chan, and Y. Niu, "Extracting causal knowledge from a medical database using graphical patterns," in *Proc. ACL*, 2000, pp. 336–343.

[7] I. Hendrickx *et al.*, "SemEval-2010 Task 8: Multi-way classification of semantic relations between pairs of nominals," in *Proc. SemEval*, 2010, pp. 33–38.

[8] J. Dunietz, L. Levin, and J. Carbonell, "The BECauSE corpus 2.0: Annotating causality and overlapping relations," in *Proc. Linguistic Annotation Workshop*, 2017, pp. 95–104.

[9] Z. Jin *et al.*, "CLadder: Assessing causal reasoning in language models," in *Proc. NeurIPS*, 2023.

[10] A. d'Avila Garcez and L. C. Lamb, "Neurosymbolic AI: The 3rd wave," *Artif. Intell. Rev.*, vol. 56, pp. 12387–12406, 2023.

[11] Corr2Cause authors, "Data generation code," https://github.com/causalNLP/corr2cause, file `code/data_gen.cpp`, accessed Oct. 2026.